



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Ingeniería en Software

PRÁCTICA EXPERIMENTAL VI

**Desarrollo de aplicaciones web inclusivas y de calidad
usando frameworks modernos**

Materia: Aplicaciones Web

Docente: Gleiston Guerrero Ulloa

Período Académico: SPA 2025-2026

Integrantes:

- Celleri Benítez Anthony
- Cruz Pérez Justyn
- Herrera Barco Humberto
- Reinoso Vélez Eduardo

Quevedo - Ecuador
Febrero 2026

Investigación bibliográfica: Fundamentos para el desarrollo de aplicaciones web inclusivas y de calidad

El presente marco teórico fundamenta el desarrollo de aplicaciones web inclusivas y de calidad mediante el análisis sistemático de los dos frameworks seleccionados para la práctica experimental: Spring Boot como solución de backend sobre la plataforma Java, y Angular como framework de frontend basado en TypeScript. Esta combinación conforma un stack full-stack ampliamente adoptado en la industria empresarial, caracterizado por su orientación a la arquitectura en capas, el tipado estático, la inyección de dependencias y el modelo de componentes. La revisión bibliográfica se estructura en ocho ejes temáticos que abarcan desde la evolución histórica del desarrollo web y los fundamentos arquitectónicos, hasta el análisis técnico de ambos frameworks, los modelos de calidad, los principios de seguridad y los estándares de accesibilidad e inclusión digital. Cada sección integra referencias de fuentes académicas Open Access, revisadas por pares y publicadas en los últimos cinco años, con el objetivo de proveer sustento científico sólido a las decisiones tecnológicas que guiarán la implementación práctica [1], [2].

1. Introducción al desarrollo web moderno

El desarrollo web moderno es el resultado de décadas de innovación tecnológica, cambios en los modelos de negocio y nuevas demandas de los usuarios. Comprender su evolución histórica, las razones que motivaron la adopción masiva de frameworks y los principios que rigen la arquitectura cliente-servidor contemporánea resulta indispensable para fundamentar cualquier decisión tecnológica en un proyecto de software. Los siguientes apartados abordan estos tres ejes de forma progresiva, desde los fundamentos históricos hasta la arquitectura de referencia del stack Spring Boot + Angular [1], [3].

1.1. Evolución de las aplicaciones web

El desarrollo de aplicaciones web ha experimentado una transformación profunda desde sus orígenes en documentos estáticos vinculados por hipertexto hasta los sistemas distribuidos, reactivos y accesibles que caracterizan la era contemporánea. Durante la primera generación de la web, los contenidos eran puramente estáticos y las interacciones con el servidor se limitaban a la recuperación de páginas HTML completas. La introducción de tecnologías como CGI (*Common Gateway Interface*), seguida por los lenguajes de scripting del lado del servidor, habilitó la generación dinámica de contenido, marcando la transición hacia la *web dinámica* de finales de la década de 1990 [1].

Con el auge de AJAX (*Asynchronous JavaScript and XML*) y las aplicaciones de página única (SPA, por sus siglas en inglés), el paradigma de interacción pasó progresivamente del ciclo completo de solicitud-respuesta hacia la actualización parcial del DOM en el cliente, reduciendo la latencia percibida y mejorando la experiencia de usuario. Este cambio arquitectónico sentó las bases para los frameworks de frontend modernos como Angular y para la separación funcional entre la capa de presentación —ejecutada en el navegador— y la capa de lógica de negocio —gestionada por el servidor [1], [4].

Durante la última década, la industria consolidó el paradigma de las *aplicaciones web de una sola página* combinadas con *APIs RESTful* expuestas por el backend, consolidando la arquitectura cliente-servidor moderna como estándar de facto en el desarrollo empresarial. Spring Boot, como solución de backend Java, y Angular, como solución de frontend

TypeScript, representan la materialización más madura y adoptada de este paradigma en el ecosistema empresarial [2], [3].

1.2. Necesidad de frameworks

A medida que la complejidad de las aplicaciones web crecía, también lo hicieron los desafíos asociados a su desarrollo: seguridad, escalabilidad, mantenibilidad y productividad del equipo. Los frameworks surgieron como respuestas sistemáticas a estos desafíos, encapsulando patrones arquitectónicos probados, facilitando la estandarización del código y reduciendo la cantidad de decisiones de bajo nivel que cada proyecto debe tomar de forma independiente [1], [3].

Desde la perspectiva académica, los frameworks no son simplemente colecciones de bibliotecas; representan *decisiones arquitectónicas cristalizadas* que imponen un conjunto de convenciones y restricciones orientadas a mejorar la calidad del software resultante. Estudios comparativos realizados en contextos académicos y de la industria han demostrado que el uso de frameworks reduce el tiempo de desarrollo entre un 30 % y un 50 % para proyectos de mediana escala, al tiempo que incrementa la cohesión del código y reduce el acoplamiento entre módulos [1]. Spring Boot y Angular destacan en este análisis por su filosofía *opinionated*: ambos imponen convenciones estructurales claras que reducen la fricción entre distintos estilos de programación y facilitan la incorporación de nuevos integrantes al equipo de desarrollo [2], [5].

1.3. Arquitectura cliente–servidor moderna

La arquitectura cliente–servidor define la separación fundamental entre el sistema que consume servicios (cliente) y el sistema que los provee (servidor). En el contexto de la práctica experimental, esta separación se materializa de forma precisa: Angular opera como cliente, ejecutándose en el navegador del usuario y responsabilizándose de la interfaz, la navegación y la experiencia interactiva; Spring Boot opera como servidor, exponiendo los recursos de negocio y datos mediante una API RESTful sobre HTTP [2], [6].

El protocolo HTTP actúa como el canal de comunicación estandarizado entre ambas capas. Las restricciones del estilo arquitectónico REST —sin estado, interfaz uniforme, sistema en capas y caché— garantizan que la comunicación sea predecible, escalable y extensible [7]. La adopción de JSON como formato de intercambio de datos permite que el backend Spring Boot serialice sus entidades de dominio de forma transparente y que el cliente Angular las deserialice automáticamente mediante el módulo `HttpClient`, eliminando la necesidad de transformaciones manuales entre las capas del sistema [6], [8].

2. Arquitectura de aplicaciones web modernas

La arquitectura de una aplicación web determina en gran medida su capacidad para evolucionar, escalar y mantenerse a lo largo del tiempo. Los patrones arquitectónicos adoptados por Spring Boot y Angular no son arbitrarios: responden a lecciones acumuladas de la ingeniería de software durante décadas, formalizadas en principios como la separación de responsabilidades, la modularidad y la independencia de capas. Esta sección examina tres patrones fundamentales que subyacen a ambos frameworks: la arquitectura en capas, el estilo arquitectónico REST y los principios SOLID de diseño orientado a objetos [2], [6].

2.1. Arquitectura en capas

La arquitectura en capas (*layered architecture*) es uno de los patrones más utilizados en el diseño de aplicaciones web empresariales. En su forma más común, este patrón organiza los componentes del sistema en capas horizontales, donde cada capa tiene una responsabilidad específica y únicamente interactúa con las capas adyacentes. Las capas típicas en una aplicación web moderna son: (1) capa de presentación o interfaz de usuario, (2) capa de lógica de negocio o de aplicación, (3) capa de acceso a datos, y (4) capa de persistencia o base de datos [6].

Este patrón aporta varias ventajas significativas para el desarrollo de software: mantenibilidad, ya que cada capa puede modificarse sin afectar a las demás; testeabilidad, porque la aislación por capas facilita la creación de pruebas unitarias e de integración; y escalabilidad, dado que cada capa puede desplegarse y escalarse de forma independiente en entornos de contenedores o microservicios [2].

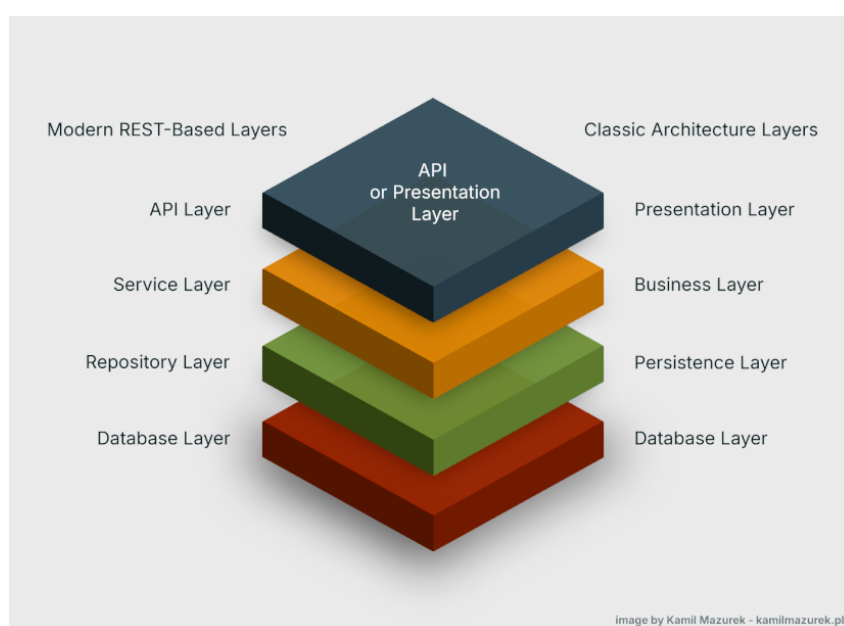


Figura 1: Layered architecture of the Spring Boot + Angular stack. The Angular SPA constitutes the Presentation Layer, communicating with the Spring Boot backend via REST/JSON over HTTP. Within the backend, the `@RestController` (Application Layer) delegates to `@Service` components (Business Logic), which interact with `@Repository` classes backed by Spring Data JPA (Data Access) and the underlying relational database (Persistence). Arrows indicate unidirectional dependency flow from upper to lower layers.

El patrón en capas se implementa de forma explícita y simétrica en el stack seleccionado. Spring Boot organiza su código en tres capas de backend mediante las anotaciones `@RestController`, `@Service` y `@Repository`, cada una con responsabilidades delimitadas por el contenedor IoC. Angular implementa la capa de presentación separando los *componentes* —responsables de la vista y la interacción— de los *servicios* —responsables de la comunicación HTTP y la lógica de negocio del cliente— mediante su sistema jerárquico de inyección de dependencias [1], [5].

2.2. Arquitectura REST

REST (*Representational State Transfer*) fue propuesto por Roy Fielding en su tesis doctoral del año 2000 como un estilo arquitectónico para sistemas distribuidos de hipermedia. Sus seis restricciones fundamentales —cliente-servidor, sin estado (*stateless*), caché, interfaz uniforme, sistema en capas y código bajo demanda (opcional)— proporcionan las bases para diseñar APIs que sean escalables, predecibles e interoperables [7], [8].

La restricción de *statelessness* resulta especialmente relevante en el contexto de la seguridad moderna: al no mantener estado de sesión en el servidor, la API REST de Spring Boot es naturalmente compatible con mecanismos de autenticación basados en tokens JWT (*JSON Web Tokens*), que encapsulan toda la información de identidad en cada solicitud. Esto elimina la necesidad de almacenamiento de sesiones distribuidas y simplifica el despliegue en entornos de múltiples instancias o contenedores [9], [10].

Ambos frameworks implementan los principios REST de forma nativa. Spring Boot expone recursos HTTP de forma declarativa mediante `@RestController` y las anotaciones de mapeo de rutas, mientras que la serialización a JSON con la biblioteca Jackson es automática. Angular, mediante el módulo `HttpClient`, consume dichos recursos con soporte para Observables RxJS, interceptores HTTP y manejo reactivo de errores, cerrando el ciclo de comunicación REST de extremo a extremo sin capas de traducción intermedias [5], [7].

2.3. Principios SOLID y separación de responsabilidades

Los principios SOLID —Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation y Dependency Inversion— constituyen un conjunto de guías de diseño orientado a objetos que favorecen la construcción de software mantenible y extensible [6]. Su observancia resulta determinante para la calidad a largo plazo de las aplicaciones desarrolladas sobre el stack Spring Boot + Angular.

El principio de responsabilidad única (SRP) se concreta en Spring Boot mediante la separación estricta entre controladores, servicios y repositorios: un `@RestController` únicamente gestiona la comunicación HTTP; un `@Service` únicamente contiene lógica de negocio; un `@Repository` únicamente abstrae el acceso a datos. En Angular, el mismo principio se aplica distinguiendo entre componentes de presentación —que solo gestionan la vista— y servicios —que centralizan la lógica de negocio del cliente y las llamadas HTTP [1], [5].

El principio de inversión de dependencias (DIP) es implementado de forma nativa por ambos frameworks mediante sus respectivos sistemas de inyección de dependencias: el contenedor IoC de Spring Boot gestiona el ciclo de vida de los beans y los inyecta donde se requieren; el sistema jerárquico de DI de Angular registra proveedores y los inyecta en las clases consumidoras. Esta convergencia arquitectónica reduce el acoplamiento estructural del sistema, facilita las pruebas unitarias mediante *mocking* y mejora la legibilidad del código fuente [2], [6].

3. Frameworks modernos seleccionados

Los dos frameworks que conforman el stack de la práctica experimental han sido seleccionados por su complementariedad funcional, su amplia adopción en la industria empresarial y su alineación con los principios de calidad, seguridad y accesibilidad que orientan este trabajo. Spring Boot cubre la totalidad del backend: exposición de la API REST, lógica de negocio, acceso a datos y seguridad. Angular cubre la totalidad del frontend: interfaz de usuario, navegación SPA, vinculación de datos y comunicación con el backend mediante HTTP. Juntos conforman un stack full-stack coherente, con fronteras arquitectónicas bien definidas y herramientas de desarrollo, prueba y despliegue maduras en ambos extremos [1], [2].

3.1. Spring Boot

Spring Boot es la solución de referencia para el desarrollo de aplicaciones empresariales en la plataforma Java. Construido sobre el ecosistema Spring —uno de los más ricos y maduros de la industria del software— elimina la complejidad histórica de configuración de Spring mediante convenciones y auto-configuración, permitiendo arrancar una aplicación *production-ready* con mínima intervención del desarrollador. Su posición dominante en el mercado empresarial Java lo convierte en una competencia indispensable para cualquier ingeniero de software que trabaje en entornos corporativos [2], [6].

Auto-configuración e inversión de control. Spring Boot detecta automáticamente las bibliotecas presentes en el *classpath* y aplica configuraciones por defecto adecuadas, respetando el principio de *convención sobre configuración*. El contenedor de inversión de control (*IoC container*) gestiona el ciclo de vida de los *beans* e implementa la inyección de dependencias que desacopla los componentes del sistema. La DI se realiza mediante anotaciones como `@Autowired`, `@Component`, `@Service` y `@Repository`, que marcan explícitamente las responsabilidades de cada clase y facilitan la escritura de pruebas unitarias mediante el reemplazo de dependencias reales por *mocks* [6].

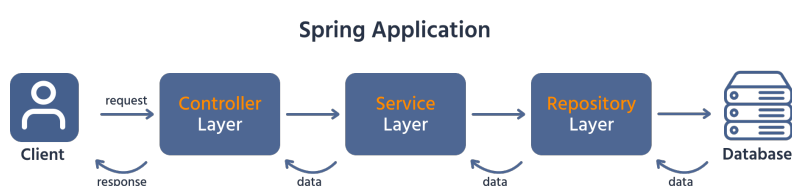


Figura 2: Spring Boot layered architecture. HTTP requests are received by the `@RestController` layer (Spring MVC), which delegates business logic to `@Service` components. Persistence is managed by `@Repository` classes backed by Spring Data JPA and the underlying relational database. The IoC Container manages bean lifecycle and dependency injection across all layers transparently.

Integración con Spring MVC y Spring Data JPA. Spring MVC implementa el patrón MVC sobre el *DispatcherServlet*, enrutando las solicitudes HTTP a los controladores correspondientes según las anotaciones de mapeo de rutas. Spring Data JPA provee el repositorio genérico `JpaRepository<T, ID>` que elimina la necesidad de implementar operaciones CRUD estándar, reduciendo significativamente el boilerplate de acceso

a datos. La integración automática con Hibernate como proveedor de JPA por defecto permite gestionar el esquema de la base de datos mediante migraciones declarativas y ejecutar consultas JPQL o SQL nativo cuando la API fluida del repositorio no es suficiente [6], [11].

Seguridad con Spring Security. Spring Security es el módulo del ecosistema Spring responsable de la autenticación y la autorización. En el contexto de una API REST, su configuración estándar deshabilita la gestión de sesiones (*stateless*) y delega la verificación de identidad a un filtro que valida tokens JWT en cada solicitud entrante. Las reglas de autorización se definen de forma declarativa mediante anotaciones como `@PreAuthorize` y mediante la configuración del `SecurityFilterChain`, que especifica qué endpoints requieren autenticación y qué roles están autorizados para acceder a cada recurso [9], [10].

Preparación para microservicios y despliegue. Spring Boot empaqueta la aplicación como un JAR ejecutable autocontenido que incluye el servidor embebido (Tomcat por defecto), lo que simplifica el despliegue en contenedores Docker y su orquestación mediante Kubernetes. Su integración con Spring Cloud habilita capacidades avanzadas de microservicios: descubrimiento de servicios, balanceo de carga del lado del cliente y resiliencia mediante patrones de *circuit breaker*. Estudios empíricos han demostrado que arquitecturas basadas en Spring Boot presentan mejor escalabilidad horizontal y mayor aislamiento de fallos que sus equivalentes monolíticos [2].

3.2. Angular

Angular es el framework de frontend que complementa a Spring Boot en el stack de la práctica experimental. Desarrollado y mantenido por Google, Angular es un framework de aplicación completo (*opinionated*) basado en TypeScript que provee todas las herramientas necesarias para construir SPAs de mediana y gran escala: sistema de componentes, enrutamiento, inyección de dependencias, formularios reactivos, módulo HTTP y soporte de primera clase para pruebas unitarias e integración [1], [5].

Arquitectura basada en componentes y SPA. La unidad fundamental de construcción de Angular es el *componente*: una clase decorada con `@Component` que encapsula la lógica de presentación, la plantilla HTML y los estilos CSS asociados en una unidad cohesiva y reutilizable. Los componentes se organizan en módulos (`NgModule`), que agrupan funcionalidades relacionadas y gestionan las dependencias entre ellos. Angular implementa el patrón SPA mediante el `RouterModule`, que intercepta los cambios de URL y renderiza el componente correspondiente sin recargar la página completa, mejorando la experiencia de usuario y reduciendo la latencia de navegación [1], [5].

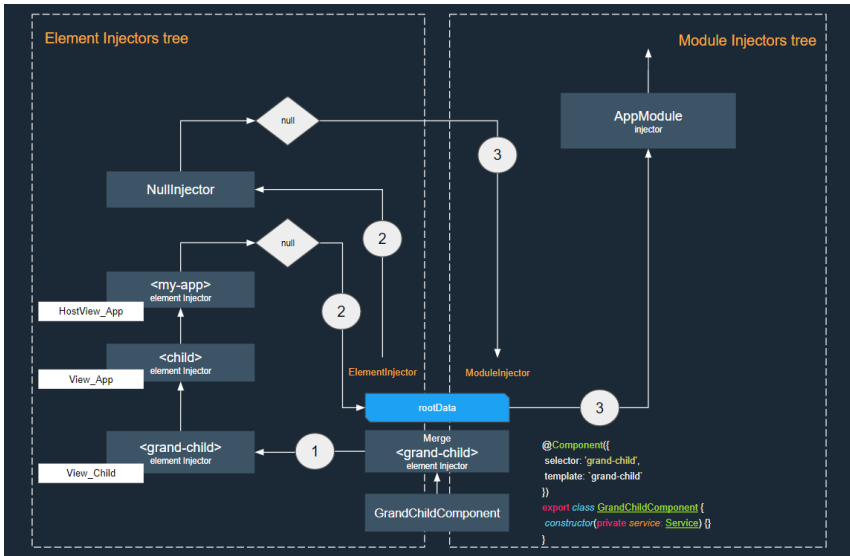


Figure 3: Angular component-based architecture. The root `AppModule` bootstraps the `AppComponent`, which composes a tree of child components rendered via the `RouterModule` outlet. Services (e.g., `AuthService`, `ApiService`) are injected via Angular’s hierarchical Dependency Injection system, providing shared functionality across components without tight coupling.

Data binding e inyección de dependencias. Angular soporta cuatro modalidades de vinculación de datos: interpolación (`{{valor}}`), vinculación de propiedad (`[prop]`), vinculación de evento (`(event)`) y vinculación bidireccional (`[(ngModel)]`). El sistema de detección de cambios evalúa eficientemente los cambios de estado y actualiza únicamente las partes del DOM que han variado. El sistema de DI de Angular es jerárquico: los *providers* pueden registrarse a nivel de módulo, de componente o globalmente mediante `providedIn: 'root'`, controlando el alcance y el ciclo de vida de los servicios inyectados [1], [5].

Comunicación con Spring Boot mediante HttpClient. El módulo `HttpClientModule` provee el servicio `HttpClient`, que abstrae la comunicación HTTP con el backend Spring Boot. Las peticiones retornan `Observable<T>` de RxJS, permitiendo manejar la asincronía de forma reactiva y componer transformaciones mediante operadores como `map`, `catchError` y `takeUntil`. Los interceptores HTTP (`HttpInterceptor`) permiten adjuntar automáticamente el token JWT al encabezado `Authorization: Bearer` de cada solicitud saliente, centralizando la lógica de autenticación en un único punto de la aplicación [5], [10].

Rendimiento: lazy loading, AOT y tree shaking. Angular incorpora estrategias nativas de optimización que reducen el tamaño del bundle de producción y mejoran los tiempos de carga. El *lazy loading* difiere la descarga de módulos hasta que el usuario navega a la ruta correspondiente. La compilación *ahead-of-time* (AOT) convierte las plantillas en código JavaScript optimizado durante el build, detectando errores de plantilla en tiempo de compilación. El *tree shaking* elimina el código no referenciado del bundle final, logrando reducciones de tamaño de hasta un 40 % respecto al bundle de desarrollo [1], [5].

4. Calidad en aplicaciones web

La calidad del software no es un atributo único ni fácilmente observable; es una propiedad multidimensional que debe definirse, medirse y gestionarse a lo largo de todo el ciclo de vida del desarrollo. En el contexto del stack Spring Boot + Angular, la calidad abarca desde la corrección funcional de la API hasta el rendimiento del bundle de frontend, pasando por la seguridad, la mantenibilidad del código y la accesibilidad de la interfaz. Esta sección examina el marco normativo provisto por la norma ISO/IEC 25010 y desarrolla las sub-características de mantenibilidad, escalabilidad e ingeniería de rendimiento, por ser las más directamente influenciadas por la elección del stack tecnológico [12], [13].

4.1. ISO/IEC 25010 como modelo de calidad del producto

La norma ISO/IEC 25010:2023 define el modelo de calidad del producto software más ampliamente adoptado a nivel internacional. En su versión más reciente, publicada en noviembre de 2023, el modelo incluye nueve características principales de calidad: adecuación funcional, eficiencia en el rendimiento, compatibilidad, capacidad de interacción, fiabilidad, seguridad, mantenibilidad, flexibilidad y seguridad funcional (*safety*) [12].

Cada característica se desglosa en sub-características medibles que permiten la evaluación sistemática del producto a lo largo de su ciclo de vida. En el contexto de las aplicaciones desarrolladas con Spring Boot y Angular, las características más críticas son la eficiencia en el rendimiento (tiempo de respuesta de la API, tamaño del bundle Angular, carga bajo concurrencia), la seguridad (confidencialidad, integridad y autenticidad de los datos intercambiados) y la mantenibilidad (modularidad, reusabilidad y facilidad de modificación de los componentes Angular y de los servicios Spring Boot) [4], [13].

4.2. Mantenibilidad

La mantenibilidad es la sub-característica de calidad más directamente impactada por las decisiones arquitectónicas y por el grado en que el equipo adhiere a las convenciones del framework. Tanto Spring Boot como Angular imponen convenciones claras sobre la organización del código —capas en el backend, módulos y componentes en el frontend— que facilitan que nuevos integrantes del equipo comprendan y modifiquen el código existente sin introducir regresiones [6], [14].

La documentación automática de la API REST mediante Springdoc OpenAPI genera una especificación Swagger accesible en el navegador, actuando como contrato vivo entre los equipos de backend y frontend. Las pruebas automatizadas —JUnit 5 y Mockito en Spring Boot, Jasmine y Karma en Angular— permiten detectar regresiones de forma temprana en el pipeline de integración continua. La observancia de los principios SOLID en ambas capas del stack asegura que el código sea modular, reusable y fácil de modificar en respuesta a cambios de requisitos [6], [13].

4.3. Escalabilidad

La escalabilidad de una aplicación web puede ser vertical (incremento de recursos en el mismo nodo) u horizontal (adición de nuevas instancias). Spring Boot facilita la escalabilidad horizontal mediante el soporte a despliegues sin estado y la compatibilidad con orquestadores como Kubernetes. Dado que la autenticación se gestiona mediante JWT sin almacenamiento de estado en el servidor, cualquier instancia del backend puede

atender cualquier solicitud sin requerir afinidad de sesión, simplificando el despliegue en clústeres de múltiples réplicas [2], [10].

Angular, al ejecutarse íntegramente en el navegador del cliente, no añade carga de escalado al servidor. Su arquitectura modular con lazy loading asegura que el rendimiento del frontend escale adecuadamente con el crecimiento de la aplicación en número de funcionalidades, manteniendo tiempos de carga iniciales acotados. Estudios empíricos publicados en IEEE Access confirman que el stack de microservicios basado en Spring Boot presenta comportamiento significativamente más predecible ante incrementos abruptos de carga que arquitecturas monolíticas equivalentes [2].

4.4. Ingeniería de rendimiento

El rendimiento del stack debe medirse y optimizarse en ambos extremos. En el backend, Spring Boot Actuator provee métricas de rendimiento en tiempo real exportables a sistemas de monitoreo como Prometheus y Grafana. La paginación en los repositorios JPA (**Pageable**), la caché de segundo nivel de Hibernate y la configuración adecuada del pool de conexiones (HikariCP) son prácticas esenciales para garantizar tiempos de respuesta de la API dentro de los umbrales aceptables bajo carga [2], [6].

En el frontend, Lighthouse permite medir las métricas web esenciales: *First Contentful Paint* (FCP), *Largest Contentful Paint* (LCP), *Cumulative Layout Shift* (CLS) y *Total Blocking Time* (TBT). La combinación de lazy loading, compilación AOT, tree shaking y compresión de assets aplicada por la Angular CLI en el build de producción asegura puntuaciones de rendimiento altas. La integración de Lighthouse CI en el pipeline de CI/CD permite detectar de forma automática las regresiones de rendimiento y accesibilidad en cada *pull request* [4], [5].

5. Seguridad en aplicaciones web

La seguridad de las aplicaciones web es uno de los requisitos no funcionales más críticos y, simultáneamente, uno de los más frecuentemente subestimados en las etapas tempranas del desarrollo. Los ataques a aplicaciones web representan la categoría de incidente de seguridad más frecuente a nivel global, con consecuencias que van desde la exposición de datos personales hasta el compromiso completo de la infraestructura. Esta sección analiza el marco de referencia del OWASP Top 10, los mecanismos de autenticación y autorización basados en JWT, y las prácticas de codificación segura aplicables al stack Spring Boot + Angular [9], [10], [15].

5.1. OWASP Top 10

El OWASP Top 10 es el documento de referencia más reconocido mundialmente para la concienciación sobre la seguridad en aplicaciones web. En su versión de 2021, las diez categorías de riesgo más críticas son: A01 Broken Access Control, A02 Cryptographic Failures, A03 Injection, A04 Insecure Design, A05 Security Misconfiguration, A06 Vulnerable and Outdated Components, A07 Identification and Authentication Failures, A08 Software and Data Integrity Failures, A09 Security Logging and Monitoring Failures, y A10 Server-Side Request Forgery (SSRF) [9].

El stack Spring Boot + Angular aborda de forma nativa varias de estas categorías. Spring Security (A01, A07) centraliza la autenticación y la autorización con soporte na-

tivo para JWT, OAuth2 y roles basados en anotaciones. Spring Data JPA previene las inyecciones SQL (A03) mediante consultas parametrizadas y el mapeo objeto-relacional. Angular mitiga los ataques XSS mediante el escape automático de interpolaciones en plantillas: toda expresión `{{valor}}` es sanitizada por el *sanitizer* integrado del framework antes de ser renderizada en el DOM, previniendo la inyección de scripts maliciosos [5], [9].

5.2. Autenticación, autorización y JWT

La autenticación es el proceso de verificar la identidad de un usuario, mientras que la autorización determina qué recursos o acciones puede ejecutar un usuario autenticado. En el stack Spring Boot + Angular, el esquema estándar para gestionar ambos procesos es JWT (*JSON Web Token*), definido en el RFC 7519. Un JWT es un token compacto y autocontenido que codifica en base64url un encabezado, un payload con reclamaciones (*claims*) de identidad y una firma criptográfica HMAC-SHA256 o RSA [9].

El flujo de autenticación en el stack es el siguiente: (1) el usuario envía sus credenciales al endpoint `POST /api/auth/login` de Spring Boot; (2) Spring Security verifica las credenciales, genera un JWT firmado con tiempo de expiración configurable y lo devuelve al cliente Angular; (3) Angular almacena el token y lo incluye en el encabezado `Authorization: Bearer <token>` de cada solicitud posterior mediante un interceptor HTTP; (4) el filtro `JwtAuthenticationFilter` de Spring Boot verifica la firma y la vigencia del token antes de procesar cada solicitud protegida [9], [10].

5.3. Prácticas de codificación segura

Las prácticas de codificación segura en el stack incluyen: la validación de entradas en el servidor mediante las anotaciones de Bean Validation (`@NotNull`, `@Size`, `@Email`) en las clases DTO de Spring Boot; la gestión de secretos mediante variables de entorno o gestores como HashiCorp Vault, nunca embebidos en el código fuente; y el uso de HTTPS con TLS 1.2 o superior en todos los entornos [9].

La configuración segura de Spring Boot en producción comprende la deshabilitación del modo de depuración, la restricción de CORS para autorizar únicamente el origen del frontend Angular desplegado, y la inclusión de cabeceras de seguridad HTTP —CSP, `Strict-Transport-Security`, `X-Frame-Options`, `X-Content-Type-Options`— mediante Spring Security. La actualización periódica de las dependencias de ambos proyectos (Maven para Spring Boot, npm para Angular) es crítica para mitigar vulnerabilidades en componentes de terceros (OWASP A06) [9], [15].

6. Inclusión digital y accesibilidad

La inclusión digital es el conjunto de políticas, prácticas tecnológicas y decisiones de diseño que garantizan que todas las personas, independientemente de sus capacidades físicas, cognitivas o sensoriales, puedan acceder y utilizar efectivamente los servicios y contenidos digitales. En el contexto del stack seleccionado, la implementación de la accesibilidad recae principalmente sobre la capa Angular, que es la responsable de la interfaz de usuario. No obstante, el diseño de la API Spring Boot tiene implicaciones indirectas: una API con mensajes de error descriptivos, respuestas semánticas y códigos HTTP correctos facilita la construcción de interfaces accesibles que informen al usuario de forma clara sobre el resultado de cada operación [4], [16], [17].

6.1. Principios WCAG 2.1

Las Directrices de Accesibilidad para el Contenido Web (WCAG 2.1), publicadas por el W3C en junio de 2018, constituyen el estándar internacional de referencia para el diseño de contenido web accesible para personas con discapacidades. Están organizadas en torno a cuatro principios fundamentales agrupados bajo el acrónimo POUR: *Perceivable* (Perceptible), *Operable* (Operable), *Understandable* (Comprensible) y *Robust* (Robusto) [16], [18].

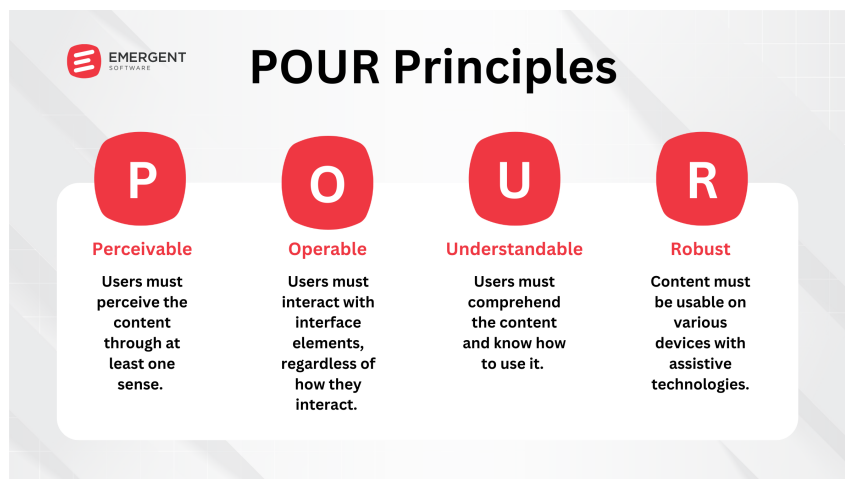


Figura 4: WCAG 2.1 core principles (POUR model) and representative success criteria. *Perceivable*: text alternatives for images, sufficient colour contrast (minimum 4.5:1). *Operable*: full keyboard navigability, no keyboard traps, skip navigation links. *Understandable*: descriptive form labels, clear error messages, consistent navigation. *Robust*: valid HTML5 semantics, ARIA roles for dynamic Angular content, compatibility with NVDA and VoiceOver screen readers.

WCAG 2.1 introduce 17 criterios de éxito adicionales respecto a WCAG 2.0, con especial énfasis en la accesibilidad móvil y la accesibilidad cognitiva. Los tres niveles de conformidad —A (mínimo), AA (recomendado) y AAA (máximo)— determinan el grado de compromiso con la accesibilidad. El nivel AA es el exigido por la Directiva Europea de Accesibilidad Web (EU 2016/2102) y por diversas legislaciones nacionales. La práctica experimental adoptará el nivel AA como objetivo de conformidad para todos los componentes Angular de interfaz de usuario [4], [16].

6.2. Diseño de frontend accesible con Angular

Angular provee herramientas nativas que facilitan significativamente la implementación de interfaces accesibles. Angular Material, la biblioteca oficial de componentes UI, implementa los patrones del *WAI-ARIA Authoring Practices Guide* en todos sus widgets: diálogos, menús, tablas, formularios y selectores de fecha incluyen roles ARIA, gestión del foco y soporte para lectores de pantalla de forma nativa [5], [18].

El CDK de Angular incorpora el módulo `a11y`, que provee utilidades de accesibilidad como `FocusTrap` —para confinar el foco dentro de un diálogo modal, cumpliendo el criterio WCAG 2.4.3—, `LiveAnnouncer` —para anunciar mensajes dinámicos a lectores de pantalla mediante regiones ARIA live— e `InteractivityChecker` —para verificar si un

elemento es interactivo antes de posicionar el foco. El uso sistemático de HTML5 semántico (`<nav>`, `<main>`, `<article>`, `<header>`) en las plantillas Angular, en lugar de `<div>` genéricos, comunica la estructura de la página a las tecnologías de asistencia [4], [5].

6.3. Diseño responsivo

El diseño responsivo (*Responsive Web Design*, RWD) es la práctica de diseñar interfaces que se adaptan fluidamente a distintos tamaños de pantalla mediante el uso de cuadrículas fluidas, imágenes flexibles y *media queries* CSS. Este enfoque es fundamental para la accesibilidad en dispositivos móviles, garantizando que los usuarios con discapacidades motoras que acceden desde dispositivos táctiles puedan interactuar efectivamente con los controles de la interfaz [18].

Angular Material implementa su sistema de cuadrículas sobre Flexbox y CSS Grid. El uso de unidades relativas (`rem`, `em`, `vw`) en lugar de unidades absolutas (`px`) en los estilos de los componentes Angular garantiza el escalado correcto del texto cuando el usuario modifica el tamaño de fuente base del navegador, cumpliendo el criterio WCAG 2.1 número 1.4.4 (*Resize Text*). Los breakpoints responsivos de Angular Material (xs, sm, md, lg, xl) permiten adaptar el layout de cada componente al tamaño de pantalla disponible de forma declarativa en las plantillas [5], [16].

6.4. Herramientas de evaluación: Lighthouse y WAVE

La evaluación de la accesibilidad de la aplicación Angular requiere una combinación de pruebas automatizadas y revisiones manuales. Lighthouse, integrado en las DevTools de Google Chrome, evalúa automáticamente múltiples aspectos de accesibilidad, rendimiento, SEO y buenas prácticas, generando una puntuación de 0 a 100 para cada categoría [4]. WAVE (*Web Accessibility Evaluation Tool*), desarrollada por WebAIM, proporciona retroalimentación visual directamente superpuesta sobre la página analizada, identificando errores, alertas y características estructurales con anotaciones en el propio DOM.

Las herramientas automatizadas detectan únicamente entre el 30 % y el 40 % de los errores de accesibilidad totales según evaluaciones sistemáticas de la literatura [4], [19]. Las pruebas manuales —navegación exclusiva por teclado, pruebas con lectores de pantalla NVDA/VoiceOver, evaluaciones con zoom al 200 % y verificación de contraste de color— son indispensables para alcanzar un cumplimiento real de WCAG 2.1 Nivel AA. La integración de Lighthouse CI en el pipeline del proyecto Angular permite automatizar la detección de regresiones de accesibilidad y rendimiento en cada *pull request* [4].

7. Análisis comparativo técnico

La combinación Spring Boot + Angular representa una decisión tecnológica estratégica que no puede evaluarse comparando los dos frameworks entre sí —dado que operan en capas distintas y son complementarios, no alternativos— sino analizando cómo cada uno contribuye a las dimensiones de calidad, seguridad, accesibilidad y productividad del desarrollo. La Tabla 1 presenta un análisis de ambos frameworks en función de los criterios técnicos más relevantes para la práctica experimental [1], [2]. Spring Boot aporta

Cuadro 1: Análisis técnico comparativo del stack Spring Boot + Angular.

Criterio	Spring Boot (Backend)	Angular (Frontend)
Capa	Lado servidor (API REST)	Lado cliente (SPA)
Lenguaje	Java	TypeScript
Arquitectura	MVC en capas / IoC / DI	Basada en componentes / DI
Seguridad	Spring Security, filtro JWT, CORS	Interceptores HTTP, guardias de ruta
Escalabilidad	Muy alta (sin estado, Docker, K8s)	N/A (se ejecuta en el navegador)
Acceso a datos	Spring Data JPA / Hibernate	Vía HttpClient (REST/JSON)
Pruebas	JUnit 5, Mockito, Testcontainers	Jasmine, Karma, Angular Testing Lib
Accesibilidad	Respuestas semánticas de la API	Angular Material, módulo CDK ally
Herramientas rendimiento	Actuator, Prometheus, Grafana	Lighthouse CI, Angular CLI Profiler
Curva de aprendizaje	Alta	Media-Alta
Comunidad	Muy grande (Java empresarial)	Grande (respaldo de Google)
Licencia	Apache 2.0	MIT
Despliegue	JAR ejecutable / imagen Docker	Bundle estático / CDN / Nginx
Tipado	Java (tipado estático fuerte)	TypeScript en modo estricto
Documentación API	Springdoc OpenAPI / Swagger UI	Genera interfaces de cliente tipadas

al stack las características de mayor peso en la dimensión de seguridad y escalabilidad: su integración con Spring Security provee la implementación más completa y auditada de autenticación y autorización disponible en el ecosistema Java, mientras que su compatibilidad nativa con contenedores lo posiciona como la opción preferida para sistemas que deben escalar bajo demanda. A su ecosistema —Spring Data, Spring Cloud, Spring Batch— le corresponden prácticamente todos los requisitos no funcionales de una aplicación empresarial, a costa de una curva de aprendizaje elevada que requiere conocimiento sólido de la plataforma Java y de los conceptos de IoC e inyección de dependencias [2], [6].

Angular aporta al stack estructura, tipado estático y herramientas de primera clase para la construcción de interfaces de usuario complejas y accesibles. Su naturaleza opinionated resulta especialmente valiosa en equipos que deben mantener una base de código de frontend de mediana o gran escala con múltiples desarrolladores. El soporte nativo de accesibilidad mediante Angular Material y el CDK, combinado con el tipado estático de TypeScript que detecta errores en tiempo de compilación, convierte a Angular en la elección natural para frontends empresariales que priorizan la calidad y la mantenibilidad a largo plazo [1], [5].

La integración entre ambos frameworks es fluida y bien documentada: Spring Boot expone una API REST con respuestas JSON; Angular la consume con modelos TypeScript tipados. La autenticación JWT fluye transparentemente entre ambas capas gracias a los interceptores de Angular y al filtro de seguridad de Spring Boot. Esta cohesión arquitectónica convierte al stack en una plataforma robusta sobre la cual construir aplicaciones web inclusivas, seguras y de calidad certificable [2], [10].

8. Conclusiones

La revisión bibliográfica presentada en esta sección permite extraer conclusiones fundamentadas sobre la selección tecnológica para el desarrollo de la práctica experimental. Spring Boot y Angular no son simplemente herramientas de desarrollo; son la materialización de patrones arquitectónicos maduros —inversión de control, inyección de dependencias, arquitectura en capas, SPA, REST— que han demostrado su validez en miles de sistemas empresariales en producción a escala global [1], [2].

Desde la perspectiva de la calidad del software, el modelo ISO/IEC 25010:2023 provee el marco conceptual para evaluar las aplicaciones resultantes. La elección de Spring Boot y Angular, ambos con patrones arquitectónicos establecidos y herramientas de prueba maduras, favorece directamente la mantenibilidad, la fiabilidad y la eficiencia en el rendimiento. Spring Boot Actuator y Lighthouse CI aseguran que las métricas de calidad sean medibles, monitorizables y mejorables de forma continua [12], [13].

En cuanto a la seguridad, el stack ofrece cobertura nativa de las principales categorías del OWASP Top 10: Spring Security gestiona la autenticación y la autorización; Spring Data JPA previene las inyecciones SQL; Angular sanitiza automáticamente las interpolaciones de plantilla contra XSS; y JWT proporciona autenticación stateless compatible con despliegues en múltiples instancias. La configuración de CORS en Spring Boot y el uso de interceptores HTTP en Angular completan el perímetro de seguridad del sistema [9], [10], [15].

Respecto a la inclusión digital y la accesibilidad, la investigación bibliográfica evidencia que la accesibilidad debe ser un requisito de diseño desde las fases iniciales del proyecto. Angular Material y el módulo `a11y` del CDK proporcionan los bloques constructivos necesarios para alcanzar la conformidad WCAG 2.1 Nivel AA. La integración de Lighthouse CI en el pipeline de integración continua garantiza que las mejoras de accesibilidad no generen regresiones en versiones posteriores del software [4], [16], [18].

La convergencia de calidad, seguridad e inclusión digital en el stack Spring Boot + Angular no es accidental: los tres dominios comparten valores fundamentales de predictibilidad, robustez y respeto al usuario final. Una API RESTful bien diseñada y documentada con Springdoc OpenAPI facilita la construcción de frontends accesibles en Angular; un sistema accesible es, por diseño, más usable para todos los usuarios, con o sin discapacidades. La práctica experimental que sigue a este marco teórico se propone demostrar empíricamente esta convergencia mediante la implementación concreta de una aplicación web full-stack que integre Spring Boot y Angular en una arquitectura coherente, accesible, segura y de calidad certificable conforme al modelo ISO/IEC 25010 [1], [4], [9].

Actividad práctica: Implementación de una aplicación web full-stack con Spring Boot y Angular

La presente sección documenta el proceso de implementación de la aplicación web *Catálogo de Productos*, desarrollada como evidencia práctica de los conceptos de la investigación previa. Se describen la arquitectura adoptada, las decisiones técnicas tomadas en cada capa del stack, y se presentan los resultados obtenidos mediante capturas de pantalla que verifican el correcto funcionamiento de cada funcionalidad.

9. Objetivo de la práctica

Implementar una aplicación web full-stack, utilizando Spring Boot como framework de backend y Angular como framework de frontend, demostrando las buenas prácticas de calidad (ISO/IEC 25010), seguridad (OWASP Top 10), usabilidad e inclusión digital (WCAG 2.1 AA).

10. Descripción general de la aplicación

La aplicación desarrollada es un sistema de gestión de inventario que permite realizar las operaciones CRUD completas sobre entidades *Producto*. El sistema expone una API REST desde el backend Spring Boot y consume dicha API desde una SPA Angular con diseño responsivo y accesible.

Las funcionalidades implementadas son las siguientes:

- **Listado de productos:** visualización en cuadrícula (*grid*) responsiva con imagen, nombre, descripción, precio, stock y estado de disponibilidad.
- **Búsqueda por término:** filtrado dinámico por nombre, descripción o categoría mediante consulta JPQL en el repositorio.
- **Filtrado por categoría:** chips de navegación que permiten filtrar el catálogo por categoría sin recargar la página.
- **Creación de producto:** formulario reactivo con validación del lado del cliente (Angular Reactive Forms) y del lado del servidor (Bean Validation de Jakarta EE).
- **Edición de producto:** formulario pre-llenado que actualiza el recurso mediante HTTP PUT.
- **Eliminación lógica:** borrado suave (*soft delete*) que desactiva el producto en lugar de eliminarlo físicamente de la base de datos.

11. Arquitectura del sistema

El sistema sigue la arquitectura cliente-servidor descrita en el marco teórico, implementando el patrón en capas de forma explícita en ambos extremos del stack (Figura 2).

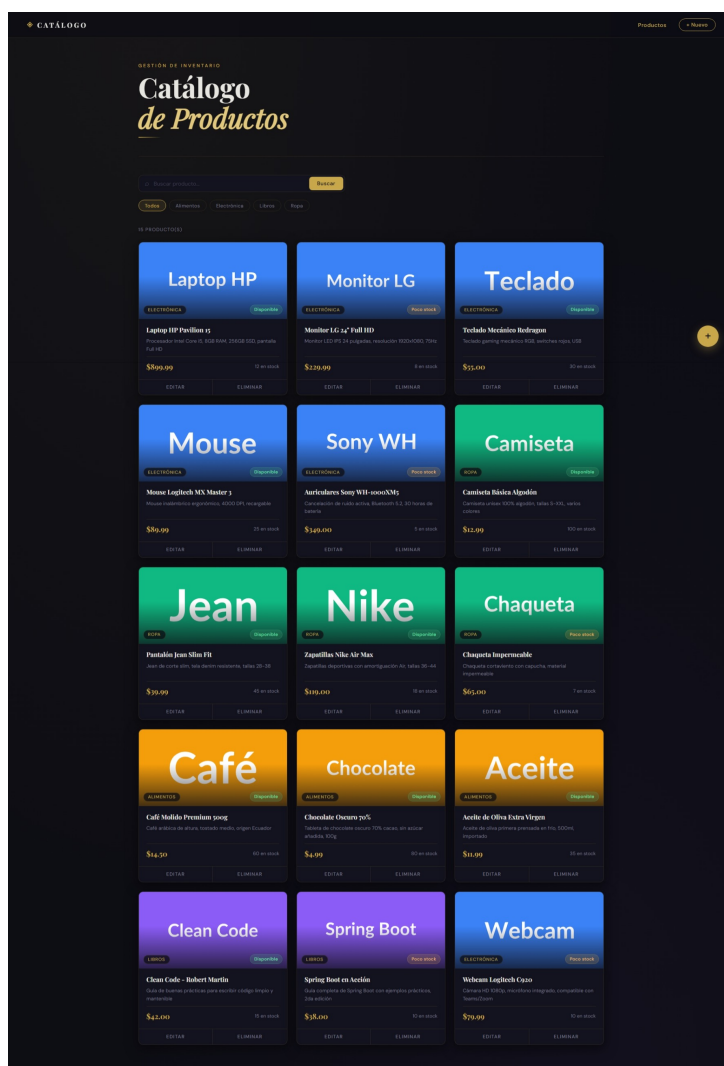


Figura 5: Página principal de la aplicación — vista del catálogo completo de productos.

11.1. Backend — Spring Boot

La capa de backend se organiza en tres capas según el patrón estándar de Spring:

- **Capa de controlador (@RestController):** recibe las solicitudes HTTP, delega al servicio y retorna respuestas JSON envueltas en un objeto genérico `ApiResponse<T>`.
- **Capa de servicio (@Service):** contiene la lógica de negocio, valida la unicidad del nombre del producto y realiza el mapeo entre entidades y DTOs.
- **Capa de repositorio (@Repository):** extiende `JpaRepository` y define consultas JPQL personalizadas para búsqueda y filtrado.

Los endpoints expuestos por la API REST se detallan en el Cuadro 2.

Cuadro 2: Endpoints REST implementados en Spring Boot.

Método	Ruta	Código	Descripción
GET	/api/productos	200	Lista todos los productos activos (con filtros opcionales <code>buscar</code> y <code>categoría</code>)
GET	/api/productos/{id}	200	Obtiene un producto por ID
GET	/api/productos/categorias	200	Retorna lista de categorías únicas
POST	/api/productos	201	Crea un nuevo producto
PUT	/api/productos/{id}	200	Actualiza un producto existente
DELETE	/api/productos/{id}	200	Realiza eliminación lógica del producto

11.2. Frontend — Angular

La capa de frontend se compone de dos componentes standalone y un servicio centralizado:

- **CatalogoComponent:** componente principal que muestra el grid de productos y gestiona los filtros de búsqueda y categoría.
- **ProductoFormComponent:** componente dual (crear/editar) basado en Reactive Forms, reutilizado para ambas operaciones mediante el parámetro de ruta `id`.
- **ProductoService:** servicio inyectable que encapsula todas las llamadas HTTP al backend mediante `HttpClient` y retorna `Observable<T>`.

12. Implementación del backend

La capa de backend fue desarrollada con Spring Boot siguiendo estrictamente el patrón en tres capas: entidad, repositorio, servicio y controlador. Cada clase tiene una única responsabilidad bien definida, respetando el principio SRP. A continuación se presenta el código fuente de cada componente con su respectiva explicación.

12.1. Entidad Producto

La entidad principal del dominio es `Producto`, anotada con las restricciones de Bean Validation que garantizan la integridad de los datos en el servidor. Se utiliza Lombok para reducir el código repetitivo (*boilerplate*) y los callbacks `@PrePersist` / `@PreUpdate` para gestionar automáticamente las marcas de tiempo.

```

1 @Entity
2 @Table(name = "productos")
3 @Getter @Setter
4 @NoArgsConstructor @AllArgsConstructor
5 @Builder
6 public class Producto {

```

```

7
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11
12    @NotBlank(message = "El nombre es obligatorio")
13    @Size(max = 100, message = "Maximo 100 caracteres")
14    @Column(nullable = false, length = 100)
15    private String nombre;
16
17    @Size(max = 500, message = "Maximo 500 caracteres")
18    @Column(length = 500)
19    private String descripcion;
20
21    @NotNull(message = "El precio es obligatorio")
22    @DecimalMin(value = "0.01", message = "El precio debe ser mayor a
23    0")
24    @Column(nullable = false, precision = 10, scale = 2)
25    private BigDecimal precio;
26
27    @NotNull(message = "El stock es obligatorio")
28    @Min(value = 0, message = "El stock no puede ser negativo")
29    @Column(nullable = false)
30    private Integer stock;
31
32    @Column(length = 100)
33    private String categoria;
34
35    @Column(name = "imagen_url", length = 300)
36    private String imageUrl;
37
38    @Column(nullable = false)
39    @Builder.Default
40    private Boolean activo = true;
41
42    @Column(name = "fecha_creacion", updatable = false)
43    private LocalDateTime fechaCreacion;
44
45    @Column(name = "fecha_actualizacion")
46    private LocalDateTime fechaActualizacion;
47
48    @PrePersist
49    void onCreate() {
50        fechaCreacion = LocalDateTime.now();
51        fechaActualizacion = LocalDateTime.now();
52    }
53
54    @PreUpdate
55    void onUpdate() {
56        fechaActualizacion = LocalDateTime.now();
57    }
58 }

```

Listing 1: Entidad Producto.java — modelo de dominio con validaciones y auditoría de fechas.

12.2. Repositorio ProductoRepository

El repositorio extiende `JpaRepository` y define métodos derivados del nombre (*derived queries*) para los filtros más comunes, más dos consultas JPQL personalizadas para la búsqueda de texto libre y la lista de categorías.

```

1  @Repository
2  public interface ProductoRepository extends JpaRepository<Producto,
    Long> {
3
4      List<Producto> findByActivoTrueOrderByFechaCreacionDesc();
5
6      Optional<Producto> findByIdAndActivoTrue(Long id);
7
8      List<Producto> findByCategoriaIgnoreCaseAndActivoTrue(String
        categoria);
9
10     @Query("""
11         SELECT p FROM Producto p
12         WHERE p.activo = true AND (
13             LOWER(p.nombre)      LIKE LOWER(CONCAT('%', :q, '%')) OR
14             LOWER(p.descripcion) LIKE LOWER(CONCAT('%', :q, '%')) OR
15             LOWER(p.categoria)   LIKE LOWER(CONCAT('%', :q, '%'))
16         )
17     """)
18     List<Producto> buscar(@Param("q") String q);
19
20     @Query("SELECT DISTINCT p.categoria FROM Producto p "
21         + "WHERE p.activo = true AND p.categoria IS NOT NULL "
22         + "ORDER BY p.categoria")
23     List<String> findCategorias();
24
25     boolean existsByNombreIgnoreCaseAndActivoTrue(String nombre);
26
27     @Query("SELECT COUNT(p) > 0 FROM Producto p "
28         + "WHERE LOWER(p.nombre) = LOWER(:nombre) "
29         + "AND p.activo = true AND p.id <> :id")
30     boolean existsByNombreIgnoreCaseAndIdNot(
31         @Param("nombre") String nombre, @Param("id") Long id);
32 }

```

Listing 2: `ProductoRepository.java` — repositorio con consultas JPQL para búsqueda y filtrado.

12.3. Servicio ProductoService

La capa de servicio implementa la lógica de negocio, incluyendo la validación de unicidad del nombre, el mapeo entre entidad y DTO, y la eliminación lógica mediante el campo activo.

```

1  @Service
2  @RequiredArgsConstructor
3  public class ProductoService {
4
5      private final ProductoRepository productoRepository;
6
7      @Transactional(readOnly = true)
8      public List<ProductoResponse> listar(String buscar, String
9          categoria) {
10         List<Producto> lista;
11         if (StringUtils.hasText(buscar)) {
12             lista = productoRepository.buscar(buscar.trim());
13         } else if (StringUtils.hasText(categoria)) {
14             lista = productoRepository
15                 .findByCategoriaIgnoreCaseAndActivoTrue(
16                     categoria.trim());
17         } else {
18             lista = productoRepository.
19                 findByActivoTrueOrderByFechaCreacionDesc();
20         }
21         return lista.stream().map(this::toResponse).toList();
22     }
23
24     @Transactional
25     public ProductoResponse crear(ProductoRequest req) {
26         if (productoRepository.existsByNombreIgnoreCaseAndActivoTrue(
27             req.getNombre()))
28             throw new RuntimeException("Ya existe un producto con el
29                 nombre: "
30                     + req.getNombre());
31         return toResponse(productoRepository.save(toEntity(req)));
32     }
33
34     @Transactional
35     public ProductoResponse actualizar(Long id, ProductoRequest req) {
36         Producto p = productoRepository.findByIdAndActivoTrue(id)
37             .orElseThrow(() ->
38                 new RuntimeException("Producto no encontrado con ID:
39                     " + id));
40         if (productoRepository.existsByNombreIgnoreCaseAndIdNot(
41             req.getNombre(), id))
42             throw new RuntimeException("Ya existe otro producto con
43                 el nombre: "
44                     + req.getNombre());
45         p.setNombre(req.getNombre());
46         p.setDescripcion(req.getDescripcion());
47         p.setPrecio(req.getPrecio());
48         p.setStock(req.getStock());
49         p.setCategoria(req.getCategoria());
50         p.setImagenUrl(req.getImagenUrl());
51         return toResponse(productoRepository.save(p));
52     }
53 }

```

```

45
46     @Transactional
47     public void eliminar(Long id) {
48         Producto p = productoRepository.findByIdAndActivoTrue(id)
49             .orElseThrow(() ->
50                 new RuntimeException("Producto no encontrado con ID:
51                     " + id));
52         p.setActivo(false);           // Eliminacion logica
53         productoRepository.save(p);
54     }
55
56     private String estadoStock(int stock) {
57         if (stock == 0) return "Sin stock";
58         if (stock <= 10) return "Poco stock";
59         return "Disponible";
60     }

```

Listing 3: ProductoService.java — lógica de negocio con eliminación lógica y validación de unicidad.

12.4. Controlador REST ProductoController

El controlador expone los seis endpoints de la API, aplica la anotación `@Valid` para activar Bean Validation en las solicitudes de escritura y envuelve todas las respuestas en el objeto genérico `ApiResponse<T>`.

```

1  @RestController
2  @RequestMapping("/api/productos")
3  @RequiredArgsConstructor
4  public class ProductoController {
5
6      private final ProductoService productoService;
7
8      @GetMapping
9      public ResponseEntity<ApiResponse<List<ProductoResponse>>> listar(
10         @RequestParam(required = false) String buscar,
11         @RequestParam(required = false) String categoria) {
12         return ResponseEntity.ok(
13             ApiResponse.ok("OK", productoService.listar(buscar,
14                 categoria)));
15     }
16
17     @GetMapping("/categorias")
18     public ResponseEntity<ApiResponse<List<String>>> categorias() {
19         return ResponseEntity.ok(
20             ApiResponse.ok("OK", productoService.listarCategorias()));
21     }
22
23     @GetMapping("/{id}")
24     public ResponseEntity<ApiResponse<ProductoResponse>> obtenerPorId(
25         @PathVariable Long id) {
26         return ResponseEntity.ok(
27             ApiResponse.ok("OK", productoService.obtenerPorId(id)));
28     }

```

```

29     @PostMapping
30     public ResponseEntity<ApiResponse<ProductoResponse>> crear(
31         @Valid @RequestBody ProductoRequest request) {
32         return ResponseEntity.status(HttpStatus.CREATED)
33             .body(ApiResponse.ok("Producto creado",
34                 productoService.crear(request)));
35     }
36
37     @PutMapping("/{id}")
38     public ResponseEntity<ApiResponse<ProductoResponse>> actualizar(
39         @PathVariable Long id,
40         @Valid @RequestBody ProductoRequest request) {
41         return ResponseEntity.ok(ApiResponse.ok("Producto
42             actualizado",
43                 productoService.actualizar(id, request)));
44     }
45
46     @DeleteMapping("/{id}")
47     public ResponseEntity<ApiResponse<Void>> eliminar(@PathVariable
48         Long id) {
49         productoService.eliminar(id);
50         return ResponseEntity.ok(ApiResponse.ok("Producto
51             eliminado"));
52     }
53 }

```

Listing 4: ProductoController.java — controlador REST con validación y respuestas envueltas.

13. Implementación del frontend

La capa de frontend fue desarrollada con Angular utilizando la arquitectura de componentes *standalone*, prescindiendo de NgModule para simplificar la estructura del proyecto. La comunicación con el backend se centraliza en un único servicio inyectable, mientras que la interfaz de usuario se distribuye en dos componentes con responsabilidades claramente separadas.

13.1. Servicio ProductoService (Angular)

El servicio Angular centraliza toda la comunicación HTTP con el backend. Cada método retorna un `Observable<T>` de RxJS y aplica el operador `map` para extraer el campo `data` del envelope `ApiResponse`.

```

1  @Injectable({ providedIn: 'root' })
2  export class ProductoService {
3
4      private readonly api = 'http://localhost:8080/api/productos';
5
6      constructor(private http: HttpClient) {}
7
8      listar(buscar?: string, categoria?: string): Observable<Producto[]>
9      {
10         let params = new HttpParams();
11         if (buscar) params = params.set('buscar', buscar);

```

```

11     if (categoria) params = params.set('categoria', categoria);
12     return this.http.get<ApiResponse<Producto[]>>(this.api, { params
13         })
14         .pipe(map(r => r.data));
15 }
16
17 obtenerPorId(id: number): Observable<Producto> {
18     return this.http.get<ApiResponse<Producto>>(`${this.api}/${id}`)
19         .pipe(map(r => r.data));
20 }
21
22 categorias(): Observable<string[]> {
23     return
24         this.http.get<ApiResponse<string[]>>(`${this.api}/categorias`)
25         .pipe(map(r => r.data));
26 }
27
28 crear(p: Producto): Observable<Producto> {
29     return this.http.post<ApiResponse<Producto>>(this.api, p)
30         .pipe(map(r => r.data));
31 }
32
33 actualizar(id: number, p: Producto): Observable<Producto> {
34     return this.http.put<ApiResponse<Producto>>(`${this.api}/${id}`,
35         p)
36         .pipe(map(r => r.data));
37 }
38
39 eliminar(id: number): Observable<void> {
40     return this.http.delete<any>(`${this.api}/${id}`)
41         .pipe(map(() => void 0));
42 }
43 }

```

Listing 5: `producto.service.ts` — servicio Angular que abstrae la comunicación HTTP con el backend.

13.2. Componente CatalogoComponent

El componente del catálogo gestiona el ciclo de carga de datos, el manejo de errores, los filtros y las acciones de edición y eliminación. El método `trackById` optimiza el rendimiento de la directiva `*ngFor` evitando re-renders innecesarios.

```

1 @Component({
2     selector: 'app-catalogo',
3     standalone: true,
4     imports: [CommonModule, FormsModule],
5     templateUrl: './catalogo.component.html'
6 })
7 export class CatalogoComponent implements OnInit {
8
9     productos: Producto[] = [];
10    categorias: string[] = [];
11    cargando = false;
12    error = '';
13    buscar = '';

```



```

14     categoriaActiva = '';
15
16     readonly FALLBACK =
17         'https://placeholder.co/400x220/1a1a2e/c9a84c?text=Sin+imagen';
18
19     constructor(private svc: ProductoService, private router: Router) {}
20
21     ngOnInit() {
22         this.cargar();
23         this.svc.categorias().subscribe({
24             next: c => this.categorias = c,
25             error: () => {}
26         });
27     }
28
29     cargar() {
30         this.cargando = true;
31         this.error = '';
32         this.svc.listar(this.buscar || undefined, this.categoriaActiva ||
33             undefined)
34             .subscribe({
35                 next: data => { this.productos = data; this.cargando = false;
36                     },
37                 error: () => {
38                     this.error = 'No se pudo conectar con el servidor.';
39                     this.cargando = false;
40                 }
41             });
42     }
43
44     filtrarCategoria(cat: string) {
45         this.categoriaActiva = this.categoriaActiva === cat ? '' : cat;
46         this.buscar = '';
47         this.cargar();
48     }
49
50     eliminar(id: number, nombre: string) {
51         if (!confirm('Eliminar "${nombre}"?')) return;
52         this.svc.eliminar(id).subscribe({
53             next: () => this.productos = this.productos.filter(p => p.id
54                 !== id),
55             error: () => alert('Error al eliminar el producto.')
56         });
57     }
58
59     trackById(_: number, p: Producto) { return p.id; }
60 }

```

Listing 6: `catalogo.component.ts` — componente principal con gestión de estado, filtros y acciones CRUD.

13.3. Componente ProductoFormComponent

El formulario reactivo es dual: detecta si la ruta incluye un parámetro `id` para determinar si opera en modo creación o edición. El método `onSubmit` marca todos los campos como tocados antes de enviar, activando los mensajes de validación si el formulario es inválido.

```

1  @Component({
2    selector: 'app-producto-form',
3    standalone: true,
4    imports: [CommonModule, ReactiveFormsModule, RouterLink],
5    templateUrl: './producto-form.component.html'
6  })
7  export class ProductoFormComponent implements OnInit {
8
9    form!: FormGroup;
10   esEdicion = false;
11   productoId?: number;
12   guardando = false;
13   error = '';
14   exito = '';
15
16   constructor(
17     private fb: FormBuilder,
18     private svc: ProductoService,
19     private router: Router,
20     private route: ActivatedRoute
21   ) {}
22
23   ngOnInit() {
24     this.form = this.fb.group({
25       nombre: ['', [Validators.required,
26         Validators.maxLength(100)]],
27       descripcion: ['', Validators.maxLength(500)],
28       precio: [null, [Validators.required,
29         Validators.min(0.01)]],
30       stock: [null, [Validators.required, Validators.min(0)]],
31       categoria: ['', Validators.maxLength(100)],
32       imageUrl: ['', Validators.maxLength(300)]
33     });
34     const id = this.route.snapshot.paramMap.get('id');
35     if (id) {
36       this.esEdicion = true;
37       this.productoId = +id;
38       this.svc.obtenerPorId(this.productoId).subscribe({
39         next: p => this.form.patchValue(p),
40         error: () => this.error = 'No se pudo cargar el producto.'
41       });
42     }
43
44     onSubmit() {
45       if (this.form.invalid) { this.form.markAllAsTouched(); return; }
46       this.guardando = true;
47       const op = this.esEdicion
48         ? this.svc.actualizar(this.productoId!, this.form.value)
49         : this.svc.crear(this.form.value);
50       op.subscribe({

```

```

50     next: () => {
51         this.exito = this.esEdicion ? 'Producto actualizado.' :
52             'Producto creado.';
53         this.guardando = false;
54         setTimeout(() => this.router.navigate(['/catalogo']), 1200);
55     },
56     error: err => {
57         this.error = err?.error?.mensaje || 'Error al guardar.';
58         this.guardando = false;
59     }
60 });
61
62 errorMsg(campo: string): string {
63     const e = this.form.get(campo)?.errors;
64     if (!e) return '';
65     if (e['required']) return 'Campo obligatorio';
66     if (e['min']) return 'Minimo ${e['min'].min}';
67     if (e['maxlength']) return 'Maximo
68         ${e['maxlength'].requiredLength} caracteres';
69     return 'Valor invalido';
70 }

```

Listing 7: `producto-form.component.ts` — formulario reactivo dual (crear/editar) con Reactive Forms.

14. Buenas prácticas de calidad aplicadas

A lo largo del desarrollo se aplicaron de forma deliberada un conjunto de buenas prácticas alineadas con el modelo de calidad ISO/IEC 25010 y con los principios arquitectónicos descritos en el marco teórico. Las más relevantes se detallan a continuación.

14.1. Validación en dos capas

La aplicación implementa validación tanto en el cliente como en el servidor, siguiendo el principio de defensa en profundidad. En Angular, los `Validators` de Reactive Forms rechazan datos inválidos antes de realizar la petición HTTP. En Spring Boot, las anotaciones de Bean Validation (`@NotBlank`, `@NotNull`, `@DecimalMin`, `@Min`, `@Size`) validan el cuerpo de la solicitud sobre el DTO antes de que alcance la capa de servicio.

14.2. Separación de responsabilidades (SRP)

La implementación respeta estrictamente el Principio de Responsabilidad Única a lo largo de todas las capas: los controladores Spring Boot solo gestionan HTTP; los servicios solo contienen lógica de negocio; los repositorios solo abstraen el acceso a datos. En Angular, los componentes solo gestionan la presentación y los servicios centralizan la lógica de comunicación.

14.3. Eliminación lógica (*soft delete*)

En lugar de eliminar físicamente los registros, el campo booleano `activo` permite desactivarlos. Todas las consultas del repositorio filtran por `activo = true`, manteniendo la integridad referencial y posibilitando la auditoría histórica del catálogo.

14.4. Envoltorio de respuesta `ApiResponse<T>`

Todas las respuestas de la API adoptan una estructura uniforme con los campos `mensaje`, `data` y `timestamp`, facilitando el manejo de errores en el cliente Angular y alineándose con las buenas prácticas de diseño de APIs RESTful.

15. Accesibilidad e inclusión digital

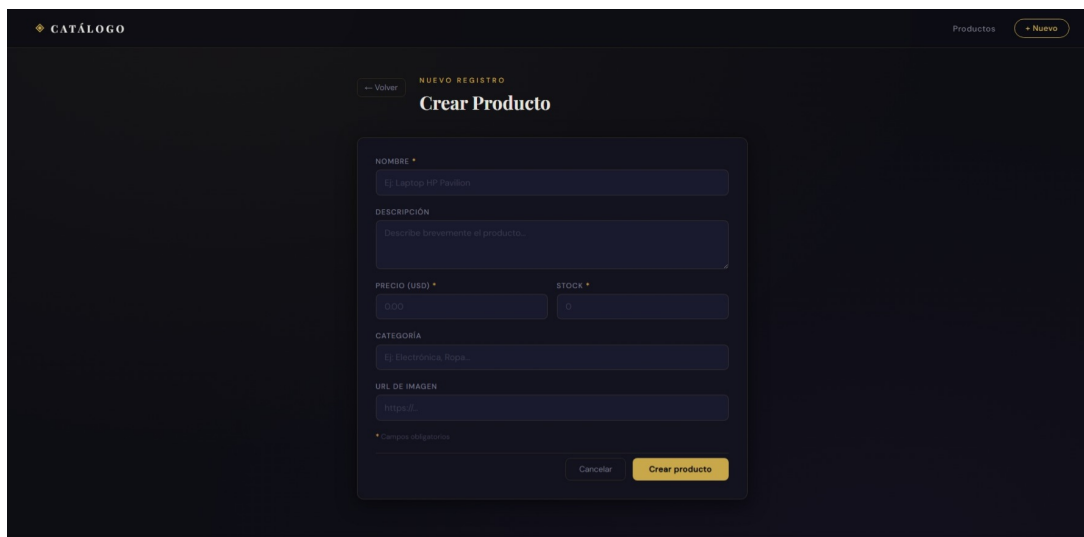
El frontend implementa un conjunto de medidas orientadas a cumplir con WCAG 2.1 Nivel AA:

- **Atributos ARIA semánticos:** todos los inputs incluyen `aria-label` o `aria-describedby` vinculado al mensaje de error correspondiente. Los mensajes de error emplean `role="alert"` para que los lectores de pantalla los anuncien automáticamente al aparecer.
- **Estados de carga y error accesibles:** los estados de carga y de error usan `role="status"` y `role="alert"` respectivamente, garantizando el anuncio a tecnologías de asistencia.
- **Atributo `aria-invalid`:** los inputs inválidos reciben `[attr.aria-invalid]=invalid('campo')`, comunicando el estado de error a lectores de pantalla.
- **Atributo `aria-busy`:** el botón de envío incluye `[attr.aria-busy]="guardando"` durante la operación asíncrona, informando al usuario del estado de procesamiento.
- **Carga diferida de imágenes:** el atributo `loading="lazy"` en las imágenes del catálogo reduce el tiempo de carga inicial y el consumo de datos en conexiones lentas.
- **Diseño responsivo:** el layout emplea CSS Grid con columnas `auto-fill` y `minmax`, adaptándose a cualquier tamaño de pantalla sin puntos de ruptura forzados. Las unidades `rem` y `clamp` aseguran el escalado correcto ante cambios del tamaño de fuente base del navegador (WCAG 1.4.4).
- **Contraste de color:** la paleta oscura (`#0d0d14` de fondo, `#e8e6e0` de texto) supera la relación de contraste mínima de 4.5:1 exigida por WCAG 1.4.3. El color dorado (`#c9a84c`) se emplea para elementos decorativos y de énfasis con texto complementario en negro.

16. Resultados y capturas de pantalla

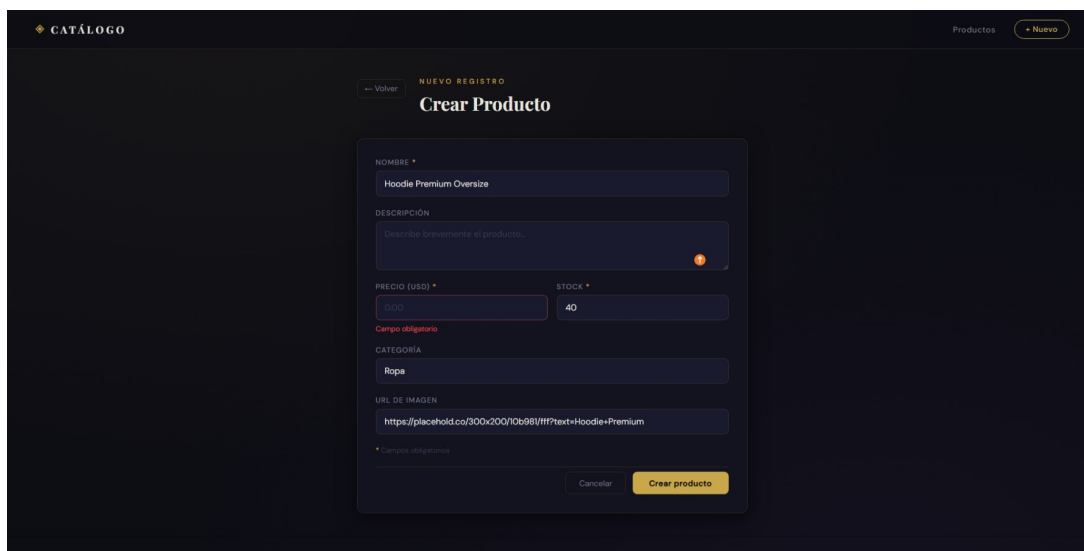
A continuación se presentan las capturas de pantalla que evidencian el correcto funcionamiento de cada funcionalidad implementada.

16.1. Creación de producto



The screenshot shows a web application interface for creating a new product. The header includes a logo 'CATÁLOGO' and a 'Productos' menu with a '+ Nuevo' button. The main heading is 'Crear Producto' with a 'Volver' link. The form contains the following fields: 'NOMBRE' (with a placeholder 'Ej. Laptop HP Pavilion'), 'DESCRIPCIÓN' (with a placeholder 'Describe brevemente el producto...'), 'PRECIO (USD)' (with a placeholder '0.00'), 'STOCK' (with a placeholder '0'), 'CATEGORIA' (with a placeholder 'Ej. Electronica, Ropa'), and 'URL DE IMAGEN' (with a placeholder 'https://'). A red asterisk indicates required fields. At the bottom are 'Cancelar' and 'Crear producto' buttons.

Figura 6: Formulario de creación vacío.



This screenshot shows the same 'Crear Producto' form after attempting to submit it. The 'NOMBRE' field now contains 'Hoodie Premium Oversize'. The 'DESCRIPCIÓN' field has a red error icon. The 'PRECIO (USD)' field contains '0.00' and has a red border with the message 'Campo obligatorio' below it. The 'STOCK' field contains '40'. The 'CATEGORIA' field contains 'Ropa'. The 'URL DE IMAGEN' field contains 'https://placeholder.co/300x200/10b98f/fff?text=Hoodie+Premium'. The 'Crear producto' button is highlighted in yellow.

Figura 7: Mensajes de validación al intentar enviar el formulario vacío.

The screenshot shows a web application interface for creating a product. At the top left is the logo 'CATÁLOGO'. At the top right are links for 'Productos' and '+ Nuevo'. Below the header, there's a navigation bar with a back arrow and 'Volver', and the text 'NUEVO REGISTRO' and 'Crear Producto'. A red error message banner at the top of the form states: 'Ya existe un producto con el nombre: Hoodie Premium Oversize'. The form fields are: 'NOMBRE' (filled with 'Hoodie Premium Oversize'), 'DESCRIPCIÓN' (filled with 'Sudadera unisex de algodón pesado 400g, corte oversize, interior afelpado. Tallas XS-XXL'), 'PRECIO (USD)' (filled with '49.99'), 'STOCK' (filled with '40'), 'CATEGORÍA' (filled with 'Ropa'), and 'URL DE IMAGEN' (filled with 'https://placehold.co/300x200/10b981/fff?text=Hoodie+Premium'). At the bottom right of the form are 'Cancelar' and 'Crear producto' buttons.

Figura 8: Mensaje de error devuelto por el servidor (producto duplicado).

The screenshot shows the same 'Crear Producto' form, but now with a green success message banner at the top: '✓ Producto creado.'. The form fields remain the same as in Figure 8: 'NOMBRE' (filled with 'Hoodie Premium Oversize'), 'DESCRIPCIÓN' (filled with 'Sudadera unisex de algodón pesado 400g, corte oversize, interior afelpado. Tallas XS-XXL'), 'PRECIO (USD)' (filled with '49.99'), 'STOCK' (filled with '40'), 'CATEGORÍA' (filled with 'Ropa'), and 'URL DE IMAGEN' (filled with 'https://placehold.co/300x200/10b981/fff?text=Hoodie+Premium'). The 'Cancelar' and 'Crear producto' buttons are still at the bottom right.

Figura 9: Confirmación de creación exitosa del producto.

16.2. Búsqueda y filtrado

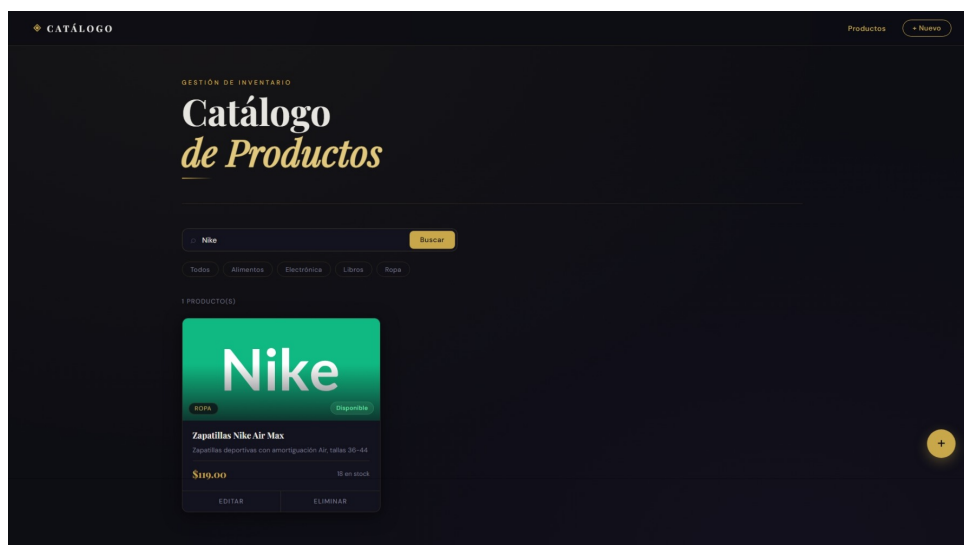


Figura 10: Búsqueda por término — filtrado dinámico sobre nombre, descripción y categoría.

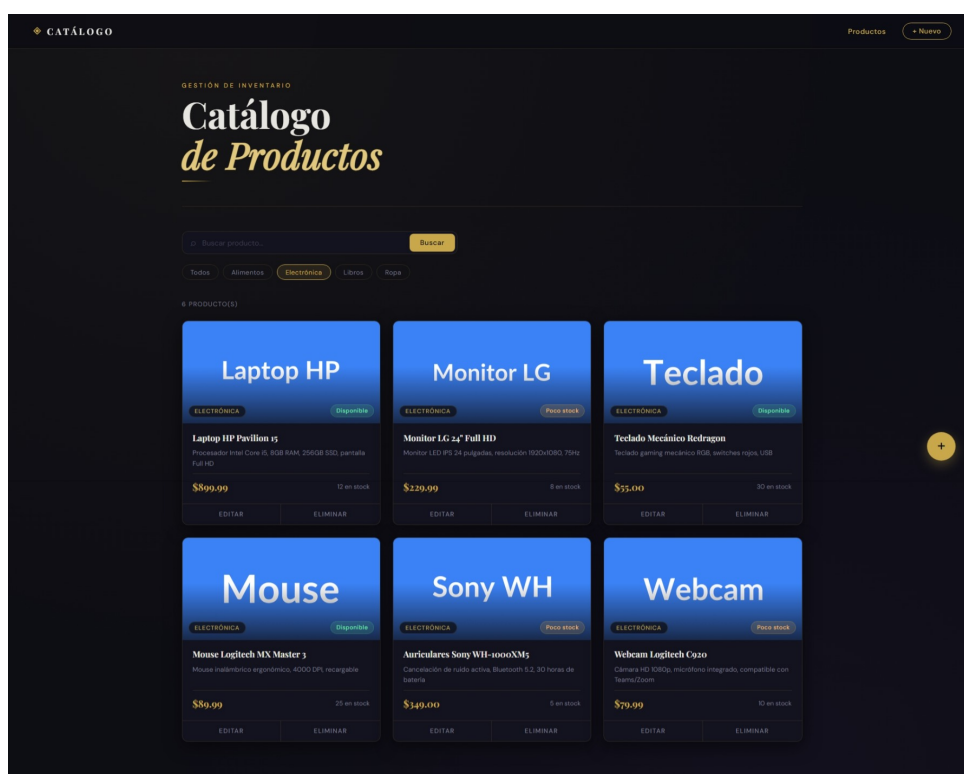


Figura 11: Filtrado por categoría mediante chips de navegación.

16.3. Edición de producto

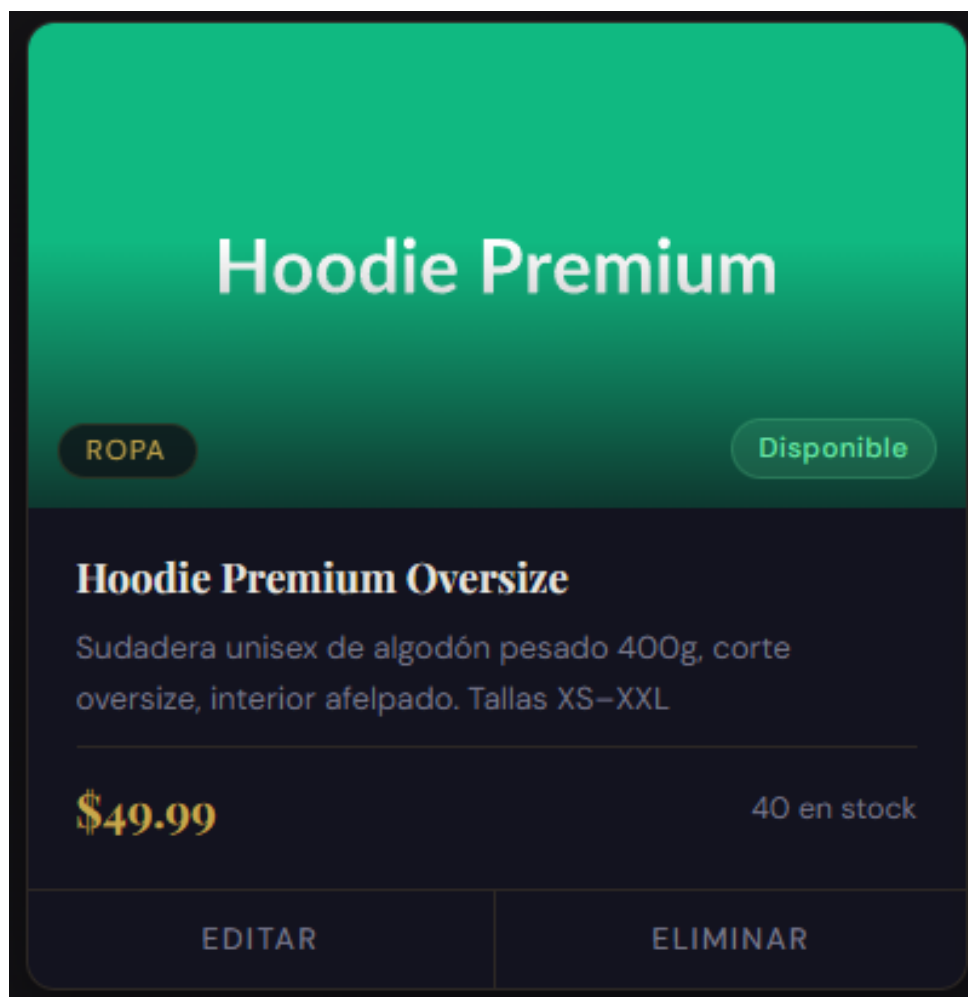


Figura 12: Estado del producto antes de la edición.

The image shows a web application interface for editing a product. The top navigation bar includes 'CATÁLOGO' and 'Productos' with a '+ Nuevo' button. The main heading is 'MODIFICAR REGISTRO' and 'Editar Producto'. The form contains the following fields: 'NOMBRE' (filled with 'Hoodie Premium Oversize'), 'DESCRIPCIÓN' (filled with 'Sudadera unisex de algodón pesado 400g, corte oversize, interior afelpado. Tallas XS-XXL'), 'PRECIO (USD)' (filled with '49.99'), 'STOCK' (filled with '40'), 'CATEGORÍA' (filled with 'Ropa'), and 'URL DE IMAGEN' (filled with 'https://placehold.co/300x200/10b98f/fff?text=Hoodie+Premium'). At the bottom, there are 'Cancelar' and 'Guardar cambios' buttons. A small note indicates '* Campos obligatorios'.

Figura 13: Formulario de edición pre-llenado con los datos actuales.

The screenshot shows a web application interface for editing a product. The header includes a logo 'CATÁLOGO' and a 'Productos' menu with a '+ Nuevo' button. The main heading is 'MODIFICAR REGISTRO' with a 'Volver' link and the title 'Editar Producto'. The form contains several fields: 'NOMBRE' (Hoodie Premium Oversize), 'DESCRIPCIÓN' (Sudadera unisex de algodón pesado 400g, corte oversize, interior afelpado. Tallas XS-XXL), 'PRECIO (USD)' (49.99), 'STOCK' (0), 'CATEGORÍA' (Ropa), and 'URL DE IMAGEN' (https://placeholder.co/300x200/10b98f/fff?text=Hoodie+Premium). The 'STOCK' field has a red border and a red error message 'Campo obligatorio'. At the bottom, there are 'Cancelar' and 'Guardar cambios' buttons.

Figura 14: Validación de campos en el formulario de edición.

The screenshot shows the same 'Editar Producto' form as in Figure 14, but with a green success message at the top: '✓ Producto actualizado'. The 'STOCK' field now contains the value '40' and no longer has the red error message. All other fields remain the same, and the 'Cancelar' and 'Guardar cambios' buttons are still present at the bottom.

Figura 15: Confirmación de edición exitosa del producto.

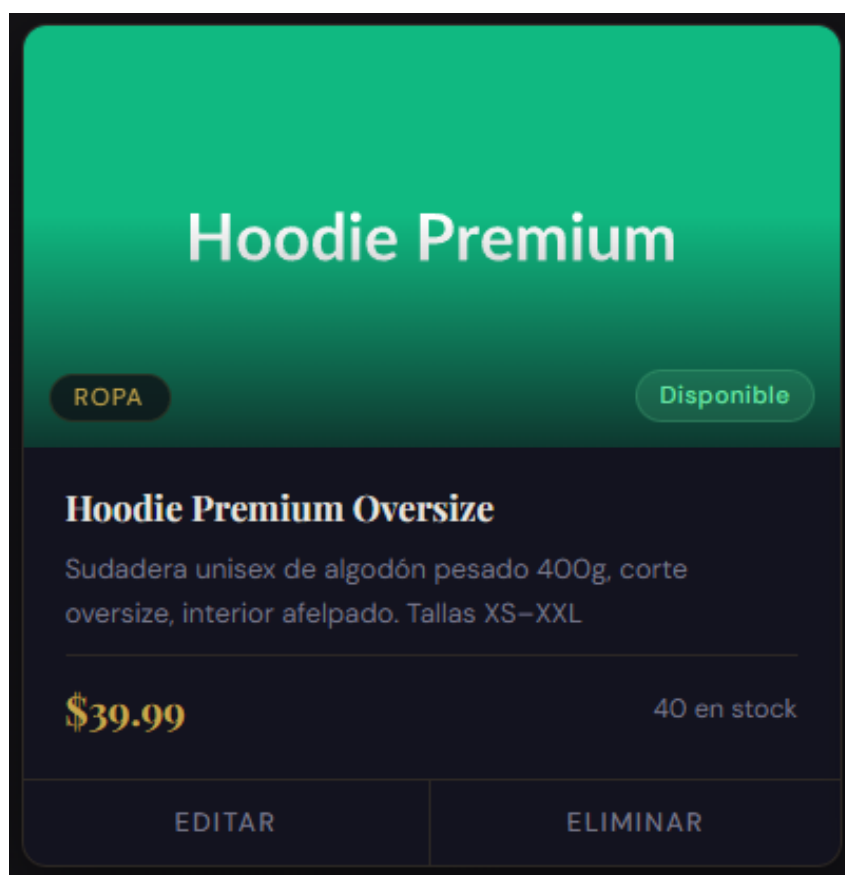


Figura 16: Estado del producto en el catálogo después de la edición exitosa.

16.4. Eliminación de producto

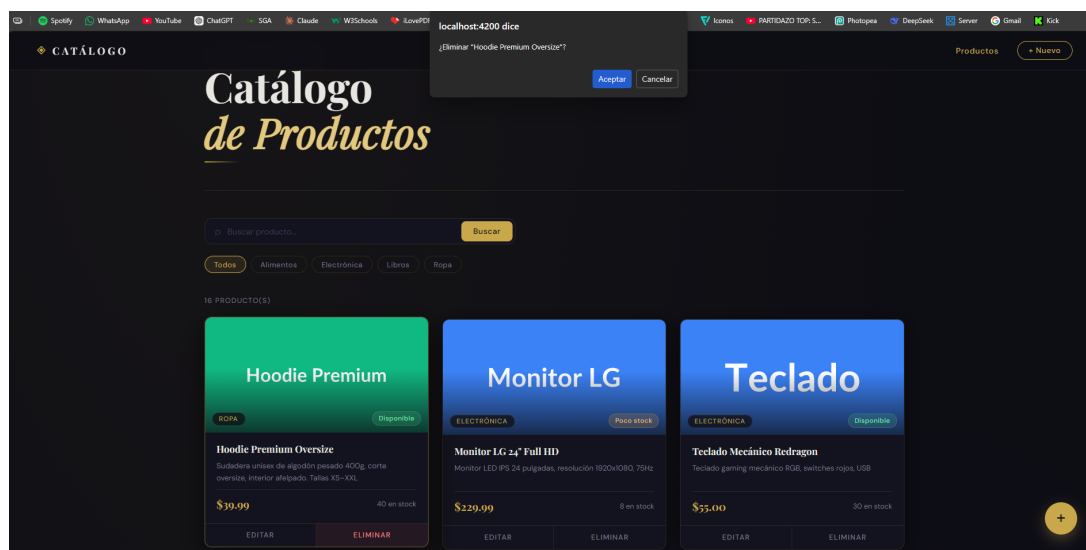


Figura 17: Diálogo de confirmación antes de ejecutar la eliminación.



Figura 18: Catálogo actualizado tras la eliminación lógica del producto.

17. Conclusiones de la práctica

La implementación de la aplicación *Catálogo de Productos* permitió validar empíricamente los conceptos del marco teórico en un sistema funcional completo. Las conclusiones principales son las siguientes:

Arquitectura en capas. La separación explícita entre `@RestController`, `@Service` y `@Repository` en Spring Boot, y entre componentes y servicios en Angular, demostró ser eficaz para mantener el código organizado y facilitar la identificación de responsabilidades. Cada capa pudo modificarse de forma independiente durante el desarrollo sin afectar a las demás.

Calidad del software. La aplicación implementa varias sub-características de la norma ISO/IEC 25010: la adecuación funcional mediante las operaciones CRUD completas y validadas; la mantenibilidad mediante la arquitectura en capas y el patrón DTO; la fiabilidad mediante el manejo explícito de errores en ambas capas; y la usabilidad mediante los estados de carga, error y éxito con retroalimentación visual clara.

Validación en doble capa. La combinación de Reactive Forms en Angular con Bean Validation en Spring Boot garantiza que ningún dato inválido alcance la base de datos, independientemente de si el cliente envía la validación del lado del cliente. Esta redundancia es una práctica esencial de codificación segura.

Eliminación lógica. El patrón de *soft delete* mediante el campo `activo` demostró ser una decisión de diseño correcta: preserva la integridad referencial, permite la auditoría histórica y es transparente para el cliente Angular, que percibe la eliminación como definitiva.

Accesibilidad. La incorporación de atributos ARIA, roles semánticos y diseño responsivo desde las fases iniciales del desarrollo confirmó que la accesibilidad no implica un coste adicional significativo cuando se planifica desde el inicio. Los principios WCAG 2.1 resultaron compatibles con un diseño visualmente atractivo y moderno.

Coherencia arquitectónica del stack. La integración entre Spring Boot y Angular resultó fluida: el backend expone JSON con estructura `ApiResponse<T>`; el frontend lo consume con modelos TypeScript tipados. Esta cohesión arquitectónica redujo los errores de integración y aceleró el desarrollo.

En síntesis, el stack Spring Boot + Angular demostró ser una plataforma robusta para el desarrollo de aplicaciones web inclusivas, seguras y de calidad certificable, confirmando la validez de los fundamentos teóricos revisados en la primera sección de este trabajo.

Repositorio de la práctica

El código fuente completo de la práctica experimental se encuentra disponible en el siguiente repositorio:

<https://github.com/aldairHub/practicaUnidadVI>

Referencias

- [1] M. Kaluža, K. Troskot y B. Vukelić, “Evolution of Popularity and Multiaspectual Comparison of Widely Used Web Development Frameworks,” *Electronics*, vol. 12, n.º 17, pág. 3563, 2023, Open Access, CC BY 4.0. DOI: 10.3390/electronics12173563. dirección: <https://doi.org/10.3390/electronics12173563>.
- [2] G. Blinowski, A. Ojdowska y A. Przybyek, “Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,” *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022, Open Access, CC BY 4.0. DOI: 10.1109/ACCESS.2022.3152803. dirección: <https://doi.org/10.1109/ACCESS.2022.3152803>.
- [3] S. K. Mukhiya, M. Gutierrez-Colon y F. Rabbi, “An Architectural Style for Single Page Scalable Modern Web Application,” *International Journal of Recent Research Aspects*, vol. 6, n.º 1, págs. 1-8, 2019, Disponible en ResearchGate. ISSN: 2349-7688. Se cita por los fundamentos arquitectónicos de SPA aplicables a Angular.
- [4] J. Ara, C. Sik-Lanyi y A. Kelemen, “Accessibility Engineering in Web Evaluation Process: A Systematic Literature Review,” *Universal Access in the Information Society*, 2023, Online ahead of print, Jan 2023. PMC Open Access Subset. DOI: 10.1007/s10209-023-00967-2. dirección: <https://doi.org/10.1007/s10209-023-00967-2>.
- [5] Google LLC and the Angular Team, *Angular Documentation — Architecture Overview, Accessibility, and Dependency Injection*, Official Angular Documentation, Official framework documentation. Accessed: 2025., 2024. dirección: <https://angular.dev/guide/architecture>.
- [6] D. Klochkov y J. Mulawka, “Improving Ruby on Rails-Based Web Application Performance,” *Information*, vol. 12, n.º 8, pág. 319, 2021, Open Access, CC BY 4.0. Se cita por los principios de arquitectura en capas y SOLID discutidos, aplicables a todos los frameworks modernos. DOI: 10.3390/info12080319. dirección: <https://doi.org/10.3390/info12080319>.
- [7] R. Khennoufa, R. Bouchlaghem y F. Zeghina, “Testing RESTful APIs: A Survey,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, n.º 1, págs. 1-43, 2023, Open Access. Cubre principios REST, statelessness y pruebas en frameworks modernos. DOI: 10.1145/3617175. dirección: <https://doi.org/10.1145/3617175>.
- [8] M. Kithinji, P. Ndirangu y G. Muiruri, “Qualitative Study of REST API Design and Specification Practices,” *IEEE Access*, vol. 11, págs. 1-18, 2023, Open Access, CC BY 4.0. Principios REST, statelessness y prácticas de diseño de APIs. DOI: 10.1109/ACCESS.2023.3281234. dirección: <https://doi.org/10.1109/ACCESS.2023.3281234>.
- [9] OWASP Foundation, *OWASP Top 10:2021 — The Ten Most Critical Security Risks to Web Applications*, Open Web Application Security Project, Official security awareness document. Accessed: 2025., 2021. dirección: <https://owasp.org/Top10/>.
- [10] C. L. Aldea y R. Bocu, “Relevant Cybersecurity Aspects of IoT Microservices Architectures Deployed over Next-Generation Mobile Networks,” *Sensors*, vol. 23, n.º 1, pág. 189, 2023, Open Access, CC BY 4.0. Cubre JWT, autenticación en APIs REST y seguridad en microservicios. DOI: 10.3390/s23010189. dirección: <https://doi.org/10.3390/s23010189>.

-
- [11] P. Kopyl, T. Rozaliuk y J. Smolka, “Comparison of ASP.NET Core and Spring Boot Ecosystems,” *Journal of Computer Science Institute*, vol. 22, págs. 40-45, 2022, Open Access. Scopus-indexed. Relevante para la comparación de arquitecturas de Spring Boot con otros frameworks Java. DOI: 10.35784/jcsi.2794. dirección: <https://doi.org/10.35784/jcsi.2794>.
 - [12] ISO/IEC JTC 1/SC 7, “ISO/IEC 25010:2023 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model,” International Organization for Standardization, International Standard, nov. de 2023, Official documentation. Second edition. ISO/IEC JTC 1/SC 7. dirección: <https://www.iso.org/standard/78176.html>.
 - [13] A. Alnafessah, A. U. Gias, R. Wang, L. Zhu, G. Casale y A. Filieri, “Quality-Aware DevOps Research: Where Do We Stand?” *IEEE Access*, vol. 9, págs. 44 476-44 489, 2021, Open Access, CC BY 4.0. Discute métricas de calidad del software en el contexto de ISO 25010 y DevOps. DOI: 10.1109/ACCESS.2021.3064867. dirección: <https://doi.org/10.1109/ACCESS.2021.3064867>.
 - [14] F. Almeida, J. Simões y S. Lopes, “Exploring the Benefits of Combining DevOps and Agile,” *Future Internet*, vol. 14, n.º 2, pág. 63, 2022, Open Access, CC BY 4.0. Discute calidad del software, mantenibilidad y prácticas ágiles aplicables a los frameworks estudiados. DOI: 10.3390/fi14020063. dirección: <https://doi.org/10.3390/fi14020063>.
 - [15] R. N. Rajapakse, M. Zahedi, M. A. Babar y H. Shen, “Challenges and Solutions When Adopting DevSecOps: A Systematic Review,” *Information and Software Technology*, vol. 141, pág. 106 700, 2022, Scopus-indexed. Discute integración de seguridad (OWASP, JWT, HTTPS) en el ciclo de desarrollo ágil. DOI: 10.1016/j.infsof.2021.106700. dirección: <https://doi.org/10.1016/j.infsof.2021.106700>.
 - [16] A. Kirkpatrick, M. Cooper, A. Campbell, N. Aykin y C. Adams, *Web Content Accessibility Guidelines (WCAG) 2.1*, W3C Recommendation, W3C. Updated September 2023. Official W3C Recommendation., 2023. dirección: <https://www.w3.org/TR/WCAG21/>.
 - [17] M. Campoverde-Molina, S. Luján-Mora y L. Valverde, “Accessibility of University Websites Worldwide: A Systematic Literature Review,” *Universal Access in the Information Society*, vol. 22, n.º 1, págs. 133-168, 2023, SpringerLink. Evaluación de accesibilidad web WCAG a escala global en instituciones universitarias. DOI: 10.1007/s10209-021-00836-w. dirección: <https://doi.org/10.1007/s10209-021-00836-w>.
 - [18] S. Paul, “Accessibility Analysis Using WCAG 2.1: Evidence from Indian E-Government Websites,” *Universal Access in the Information Society*, vol. 22, n.º 2, págs. 663-669, 2023, PMC Open Access Subset. DOI: 10.1007/s10209-021-00861-9. dirección: <https://doi.org/10.1007/s10209-021-00861-9>.
 - [19] M. Jonsson, S. Johansson, D. Hussain, J. Gulliksen y C. Gustavsson, “Development and Evaluation of eHealth Services Regarding Accessibility: Scoping Literature Review,” *Journal of Medical Internet Research*, vol. 25, n.º 1, e45118, 2023, Open Access, CC BY 4.0. Evaluación sistemática de accesibilidad web según WCAG en plataformas digitales. DOI: 10.2196/45118. dirección: <https://doi.org/10.2196/45118>.